

Technical Design – Candidate Data Transformation Pipeline

Pipeline: Detect Input → Parse Recruiter CSV & GitHub JSON → Normalize (name, email, phone, skills) → Merge records using candidate_id → Remove duplicate skills → Calculate confidence score → Add provenance → Validate → Export final_candidates.json → Display results in Flask dashboard.

Canonical Output Schema: candidate_id, full_name, emails, phones, location, current_company, title/headline, skills, links, bio, github_name, years_experience, experience, education, provenance, confidence.

Normalization: Emails converted to lowercase, phone numbers standardized, duplicate skills removed, skill names normalized (e.g., SQL, HTML, API), missing values displayed as N/A.

Merge / Conflict Resolution: Candidate records are matched using candidate_id. Recruiter CSV is treated as the primary source for personal details, while GitHub contributes bio, GitHub name, location, and additional skills. Duplicate skills are removed.

Confidence Assignment: A project-defined confidence score is calculated based on the completeness of important profile fields such as email, phone, skills, company, title, location, bio, and GitHub information.

Runtime Output Configuration: The pipeline validates uploaded files, merges available data, creates a canonical JSON profile, and presents the transformed data through a Flask web dashboard.

Edge Cases Handled: Missing email/phone/location, duplicate skills, inconsistent skill capitalization, missing GitHub information, NaN values converted to N/A, and incomplete candidate profiles.

Future Improvements: Support LinkedIn and resume parsing, automatic years of experience extraction, configurable confidence weights, and richer validation rules.